fes-f2-gpu: the Rust-side preparation cost of a call, before and after a direct pack

Engineering note, not a result on the index-calculus cost. Wall times are a practicality note (AGENTS.md section 6); the counted cost is unchanged. All figures are unverified assertions of the producing session [pcc:unverified].

Question

The 2026-11-13 end-to-end report names the per-call overhead of the `fes-f2-gpu` engine as the next thing to cut: 1.0, 1.2 and 3.6 ms per call at 24, 26 and 30 unknowns against kernel times of about 0.1, 0.2 and 1.3 ms. Where does the Rust side of a call spend its time, and what does it cost to build the system the worker takes?

Method

`FES_GPU_TIMING=1` (this change) prints the per-call milliseconds of each Rust-side phase of the `fes-f2-gpu` solver: form conversion, packing, worker round trip, Gray-rank sort, and CPU verification of every point. `ic bench --sweep` ran the two Koblitz sweeps (`koblitz-n19-gpu.json`, 20 unknowns, 786 solver calls; `koblitz-n23-gpu.json`, 24 unknowns, 1878 solver calls) with the host-emulation serve worker `gpu/fes/fes_solve`, so no GPU is involved and the round trip is the CPU worker, not a GPU. Two binaries of the same source, differing only in the change below: before (`ic-timers-before`, sha256 `a6a31a51...`) and after (`target/release/ic`, sha256 `357d1eab...`). Three runs per arm and instance, arm order alternating per pair (`run_ab.sh`), `--rho-runs 2`. The preparation cost is forms plus pack before, and the direct pack alone after. Host: the authoring container, 4 logical CPUs, isolation unstated.

Change. The form path built one `QuadraticForm` (a `Vec<Vec<bool>>`) per equation, converted a second time when an equation is narrower than the system, then packed all of them into the worker's words. `fes_pack_direct` packs the same words in one pass over the monomials and reports the same two counted charges; it declines anything the form path treats differently, and that path then decides.

Results

instance	before, ms/call	after, ms/call	saved, ms/call	saved
Koblitz n=19, 20 unknowns	0.1695	0.0048	0.1647	97.1%
Koblitz n=23, 24 unknowns	0.2784	0.0074	0.2710	97.3%

Mean of three runs per arm; min to max in the figure. The other Rust-side phases are small: sorting is below 0.001 ms per call and CPU verification 0.0025 ms (n=19) and 0.0060 ms (n=23), so the form conversion was about 90 percent of the Rust-side time outside the round trip.

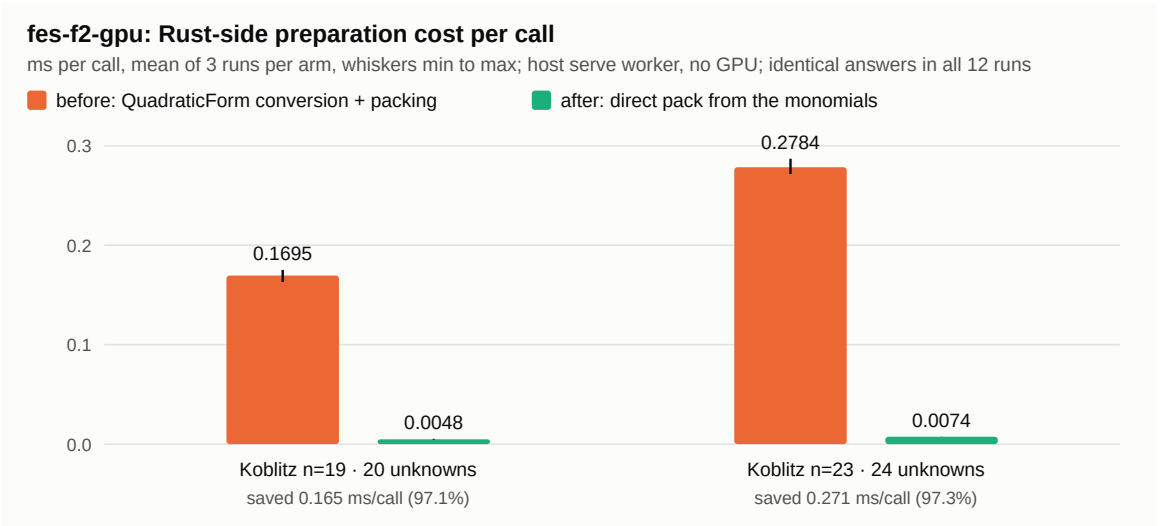


Figure 1: Preparation cost per call, before and after.

Answers unchanged. Every run's result JSON was compared with the first before-run, ignoring wall-clock fields, the per-run machine calibration and the binary hash, with floats compared to 1e-9 relative and everything else exactly: 0 of 6 runs differ at either instance (`raw/identity.txt`). Solver calls (786, 1878) and points per call (2.19, 1.96) are the same in every run. A unit test checks the packed words and both charges against the form path on 90 random systems; the 183 `ic_framework` tests pass with the host serve worker.

What this does not show

- **No GPU number.** The worker round trip here is the CPU emulation (about 0.5 ms at 20 unknowns, 6.0 ms at 24). The saving is in the Rust side, so it carries to a GPU run, but the whole-call speedup there is not measured. As an arithmetic estimate only, 0.27 ms is about a quarter of the 1.0 ms per call reported for 24 unknowns on an A100; that run used a different CPU.
- **No counted-cost effect.** S, S / rho and the gap to the matched rho do not move; this is not an improvement in the repository's metric.
- One host, three runs per arm, no independent replay.

Next

The same A/B on a GPU worker (A100) with `FES_SERVE_TIMING=1` for the worker's side of the round trip, then the cuts that those numbers point to (per-request host-to-device copies, the text protocol).

Files

`run_ab.sh`, `analyze.py`, `results.json`, `raw/timings.tsv`, `raw/identity.txt`, `raw/binaries.sha256`, sweeps `koblitz-n19-gpu.json` and `koblitz-n23-gpu.json`.